

1. 命令式范式例子

实在做不出来用命令式编程不过分吧 ()

```
1  (* 逐列放置 n 个皇后, 给出所有合法的放法 将皇后的位置表示为一个从 1 到 N 的数字列表。例如: [4; 2; 7;
   3; 6; 8; 5; 1] 表示第一列的皇后在第四行, 第二列的皇后在第二行 *)
2  let n_queen (n:int) : int list list =
3      let board = Array.make_matrix n n 0 in
4      let ans = ref [] in
5      let diag1 = Array.init (2 * n) (fun _ → 0) in
6      let diag2 = Array.init (2 * n) (fun _ → 0) in
7      let check row col =
8          let ok = ref true in
9          for i = 0 to col-1 do
10             if board.(row).(i) = 1 then ok := false
11             done; (* 注意语句间的分号 *)
12             if diag1.(row+col) = 1 || diag2.(row-col+n) = 1 then ok := false;
13             !ok
14         in
15         let rec dfs (cur:int) (path:int list) =
16             if cur = n then
17                 ans := (List.rev path) :: !ans
18             else
19                 for i = 0 to n - 1 do
```

```
20      (* 使用 begin end 包裹过程块 *)
21      if check i cur then begin
22          board.(i).(cur) ← 1;
23          diag1.(i+cur) ← 1;
24          diag2.(i-cur+n) ← 1;
25          dfs (cur + 1) ((i+1) :: path);
26          board.(i).(cur) ← 0;
27          diag1.(i+cur) ← 0;
28          diag2.(i-cur+n) ← 0;
29      end
30  done
31  in
32  dfs 0 [];
33  !ans
34
```

Fence 1

2. Dune 的使用 (更直接查看移入规约冲突方法)

```
1 (library
2   (name interpreterlib)
3   (modules ast lexer parser))
4
5 (rule
6   (targets parser.ml parser.mli parser.output)
7   (deps parser.mly)
8   (action
9     (chdir %{workspace_root}
10      ;; 提供 .output 文件在 ./_build/default/lib/parser.output
11      (run ocaml yacc -v lib/parser.mly))))
12
13 (rule
14   (targets lexer.ml)
15   (deps lexer.mll)
16   (action
17     (chdir %{workspace_root}
18      (run ocamllex -o lib/lexer.ml lib/lexer.mll))))
19
```

Figure 1

.output 文件会告诉你在哪个状态存在 **移入规约冲突**, 这样你可以相应调整 **mly** 文件的优先级/结合性, 以解决该冲突:

20: shift/reduce conflict (shift 13, reduce 7) on PLUS
state 20

```
expr : expr . PLUS expr (6)
expr : IF expr THEN expr ELSE expr . (7)

PLUS  shift 13
EOF   reduce 7
RPAREN reduce 7
THEN  reduce 7
ELSE  reduce 7
```

State 20 contains 1 shift/reduce conflict.

Figure 2

rule 的语法如下:

```
1 (rule
2   (action <action>)
3   <optional-fields>)
```

Fence 2

<action> 是你用来从依赖项生成目标的操作。

<optional-fields> 包括:

- `(target <filename>)` 或 `(targets <filenames>)` 是一个文件名列表（如果使用 `targets` 定义），或者是一个确切的文件名（如果使用 `target` 定义）。Dune 需要静态地知道每个规则的目标。如果 `(targets)` 可以从操作中推断出来，则可以省略。
- `(deps <deps-conf list>)` 指定了规则的依赖关系。

`library` 的语法如下：

```
1 (library
2   (name <library-name>)
3   <optional-fields>)
```

Fence 3

- `name` 为 `foo` 的库的模块将作为 `Foo.XXX` 在 `foo` 之外可用
- `(modules <modules>)` 指定哪些模块是库的一部分。默认情况下，Dune 会使用与 `dune` 文件相同的目录中的所有 `.ml/.re` 文件。这包括文件系统中存在的文件以及由用户规则生成的文件。您可以通过使用 `(modules <modules>)` 字段来限制此列表。

`executable` 的语法如下：

```
1 (executable
2   (name <name>)
3   <optional-fields>)
```

Fence 4

`<name>` 是一个包含可执行文件主入口点的模块名，你只需要指定 **入口点**（哪里开始执行）

`(modules <modules>)` 指定 Dune 在构建此可执行文件时应考虑的当前目录中的哪些模块。此处未列出的模块将被忽略，并且不能在当前节描述的可执行文件内部使用。它与库的 `(modules ...)` 字段以相同的方式解释。

`(modes (<modes>))` 设置链接模式。默认是 `(exe)`。在 Dune 2.0 之前,它曾经是 `(byte exe)`

3. earlybird 插件的调试配置 (断点)

在 `bin/dune` 中:

```
1 (executable
2   (public_name test_bc)
3   (name main)
4   (libraries test_bc)
5   (modes byte exe) // 这里是关键
6 )
```

Fence 5

在 `dune-project` 中:

```
1 (lang dune 3.17)
2
3 (name test_bc)
4
5 (generate_opam_files true)
6
7 (map_workspace_root false) // 这里是关键
```

Fence 6

打断点:

- ```
1 let () = print_endline "Hello, World!";
2 | Printf.printf "%d\n" 1;;
3
```

Figure 3

`dune clean && dune build` 后开始调试 (右键 `test_bc/_build/default/bin/main.bc` )

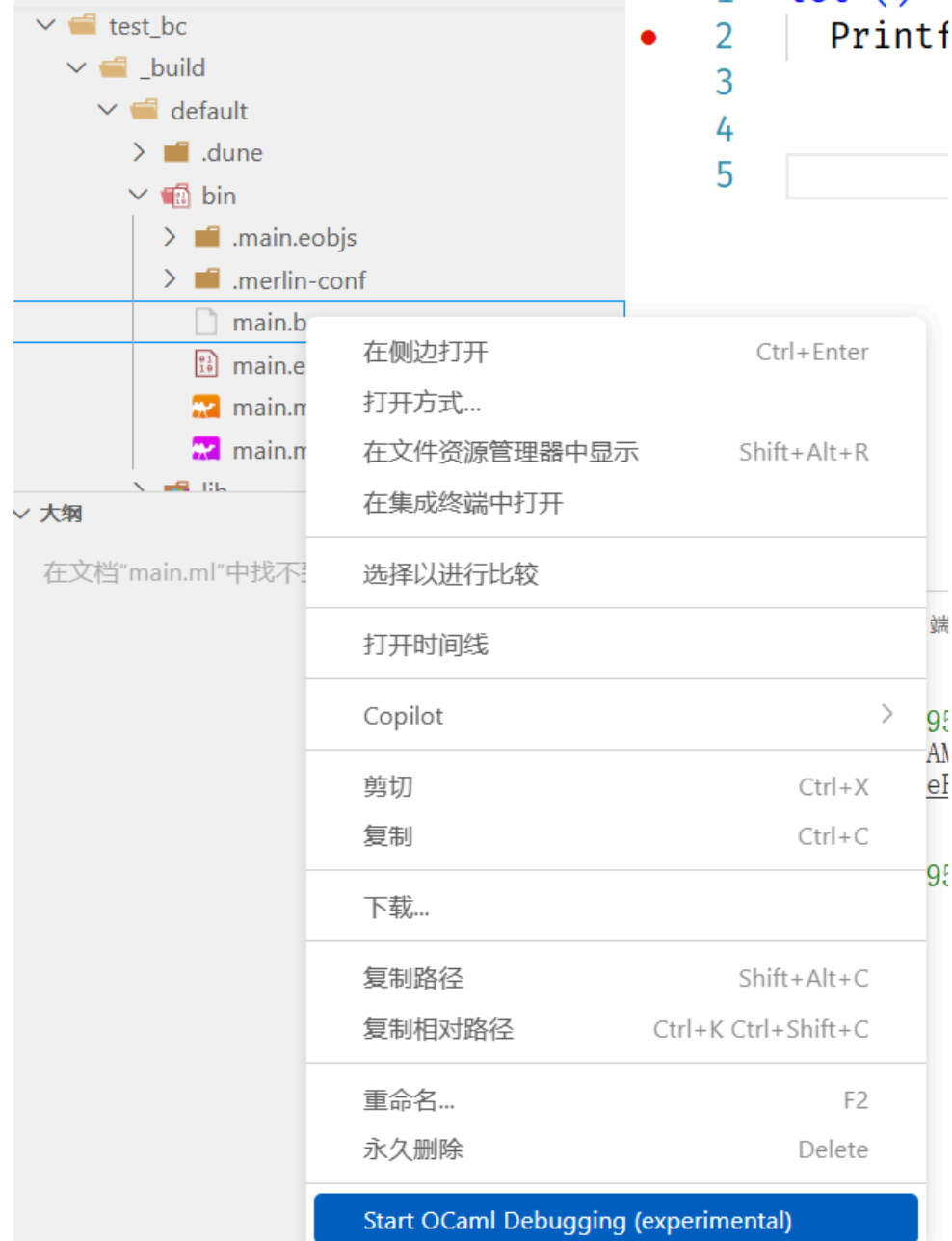


Figure 4



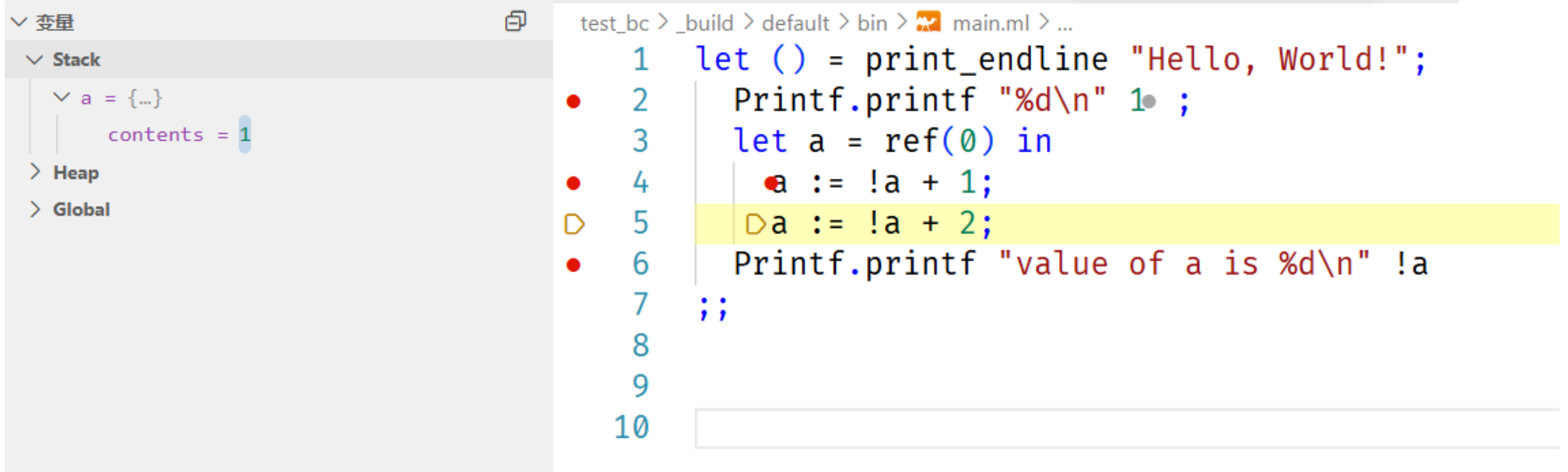


Figure 5

运行后自动切换目录到 `test_bc/_build/default/bin` , 要 `cd ../../../../` 切换回来。